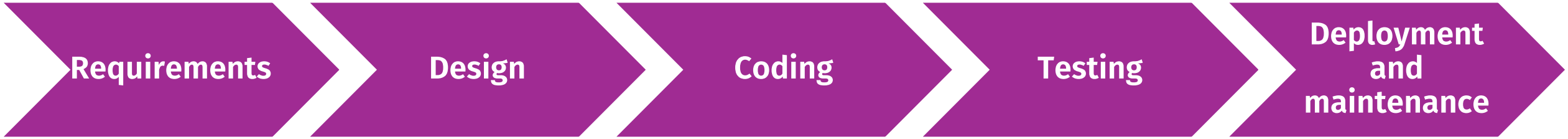
SENG 201
Software Engineering Foundations

[Week 1, Wednesday]

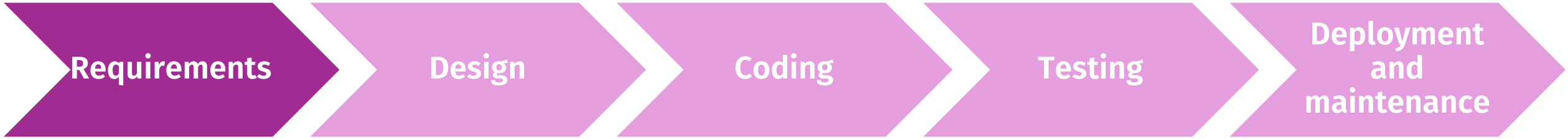


Yesterday…



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Yesterday…



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Designing Software



→ Levels of abstraction

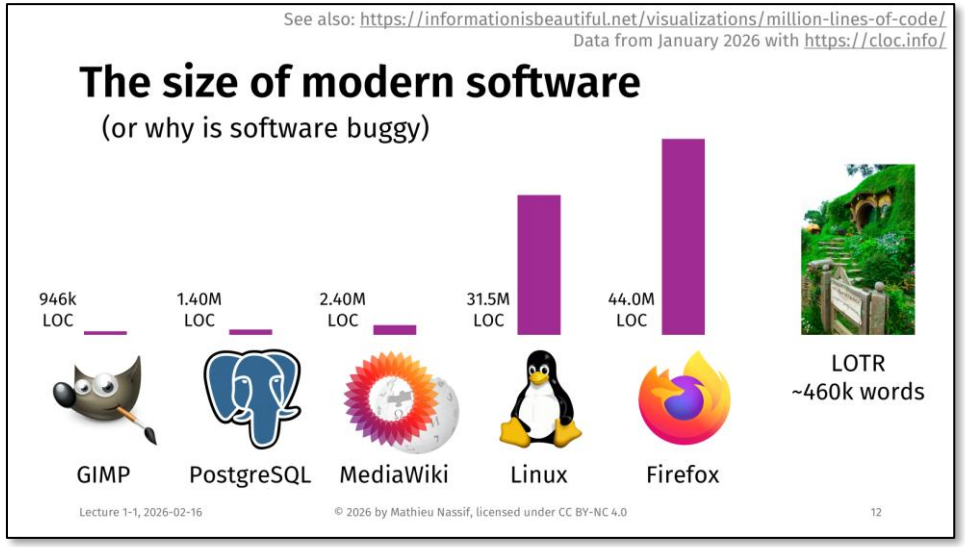
→ Design aspects

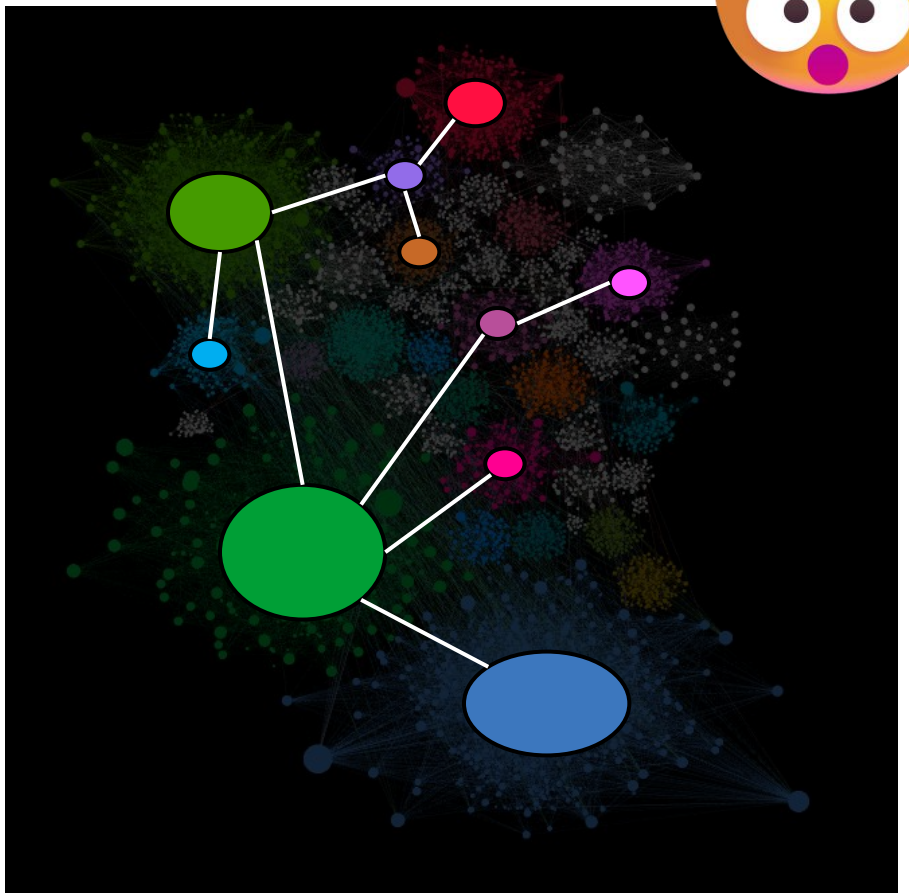
→ OOP, classes, objects

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

The purpose of design

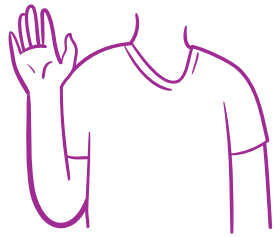
Create simplified models
to highlight high-level characteristics
without distraction from low-level details.





https://wiki-insights.epfl.ch/wikitrends/

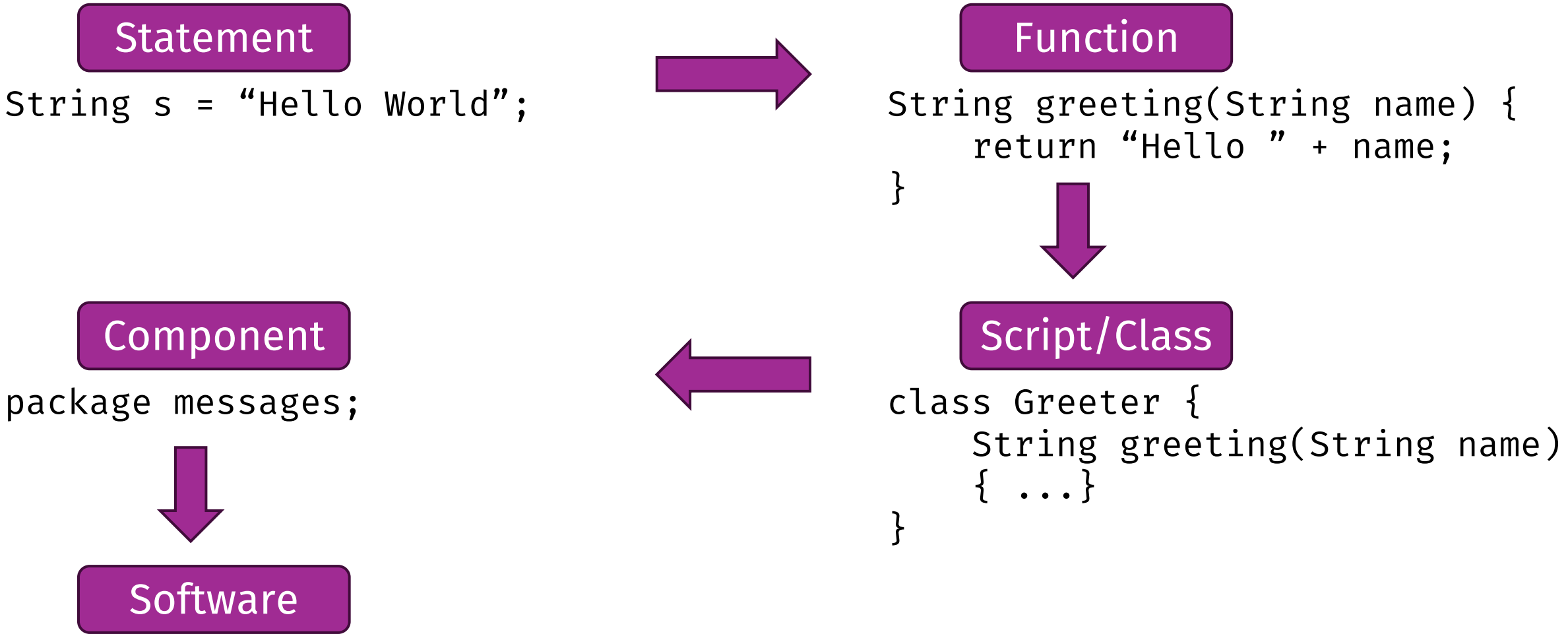
© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0



What high- and low- “levels” are we talking about?

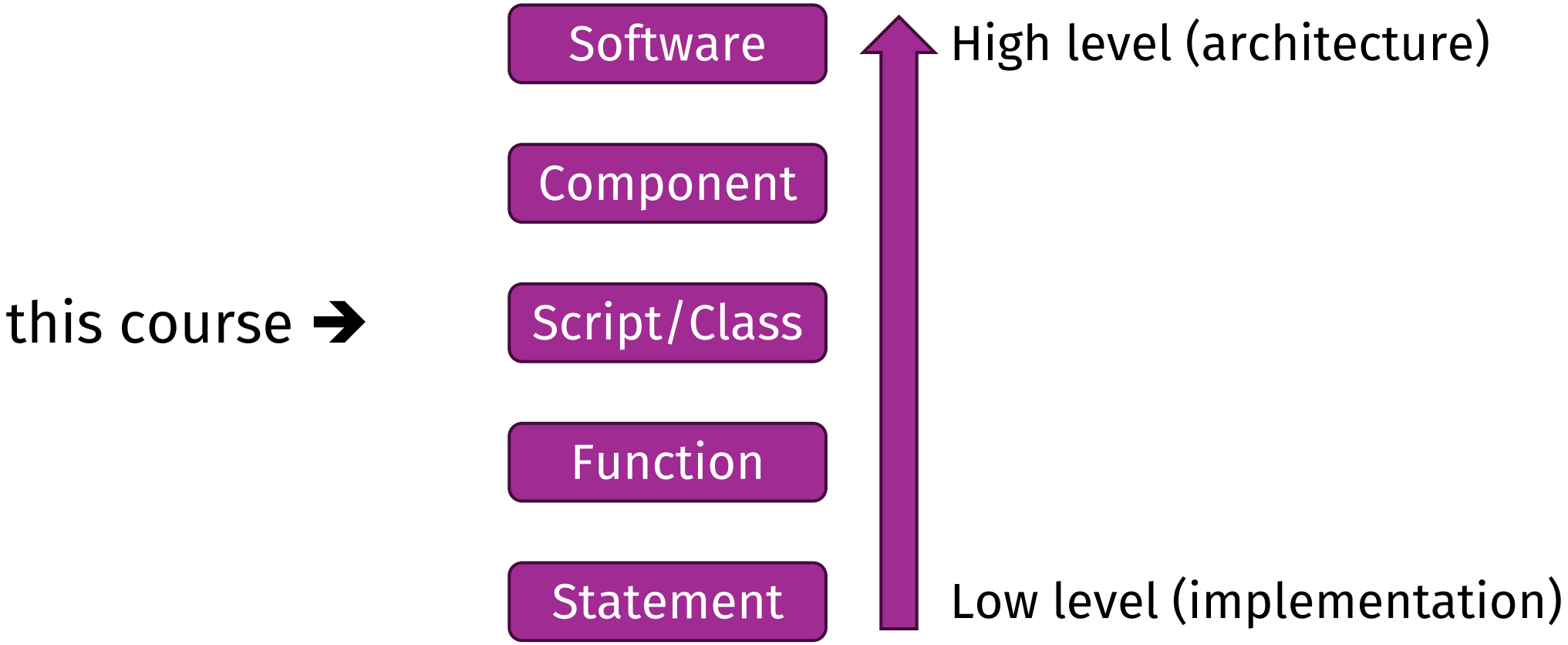
© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Levels of abstraction



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Levels of abstraction



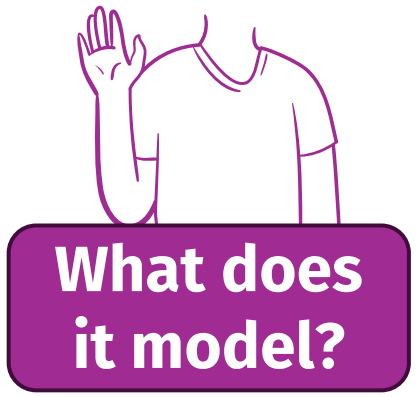
© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

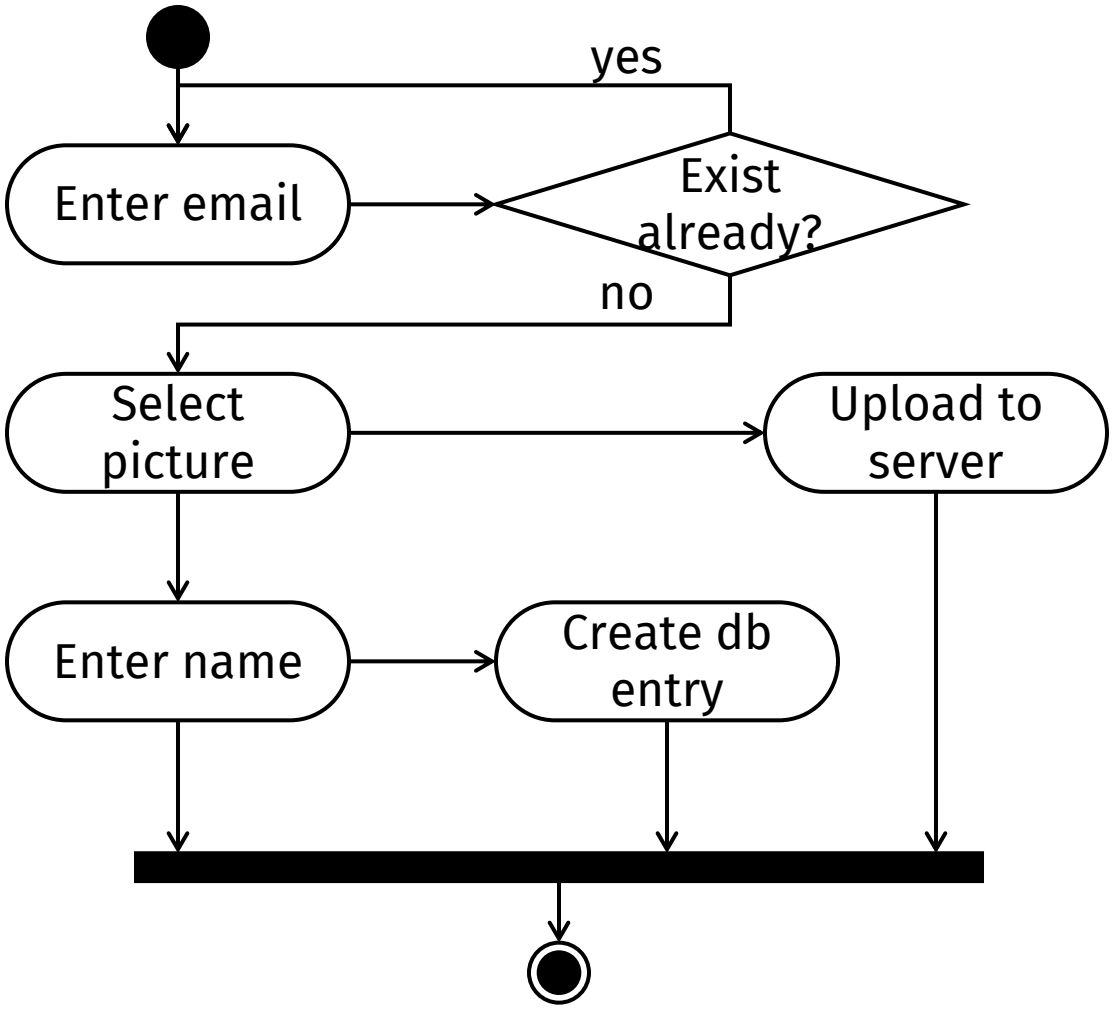
What to design?

Structure	Behaviour
• Names and responsibilities	• Algorithms
• Interaction between units	• Functional requirements
• Interfaces	• Valid/invalid states
• Data/memory representation	• Communication sequences

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

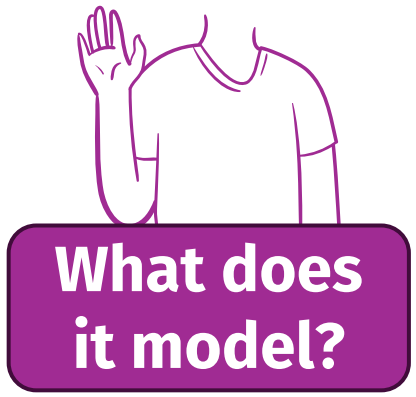
Example 1: User account

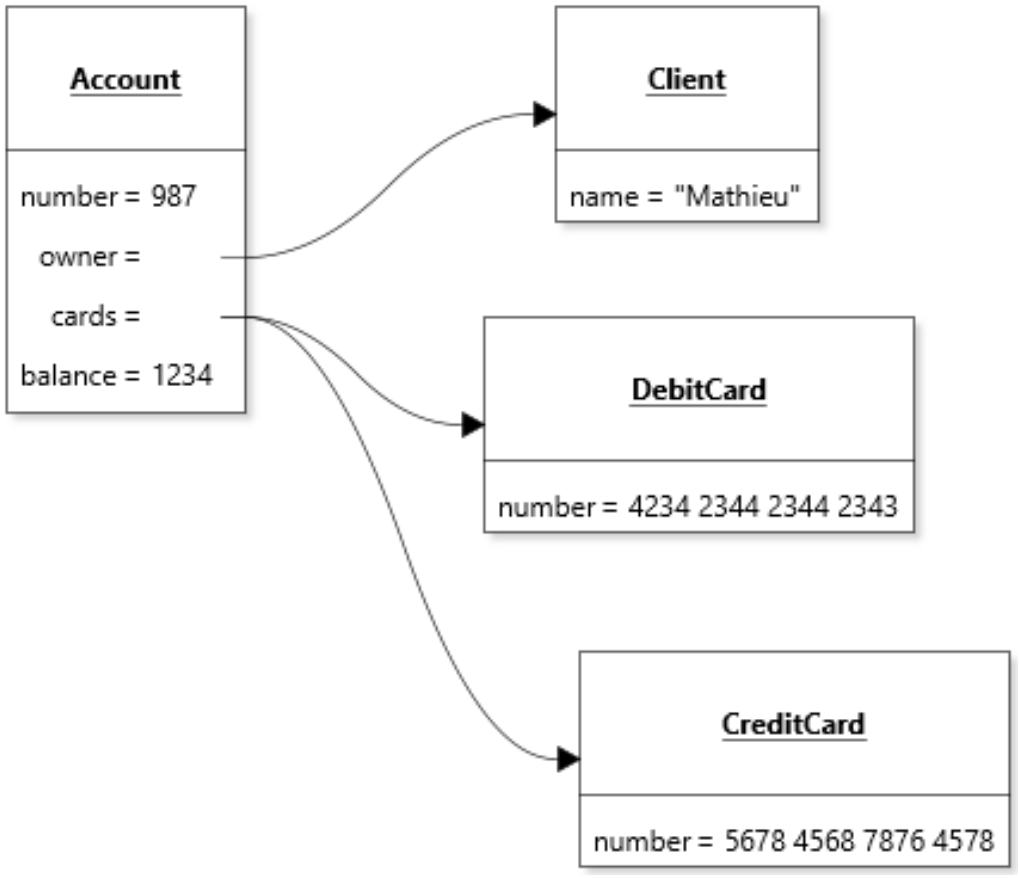




© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

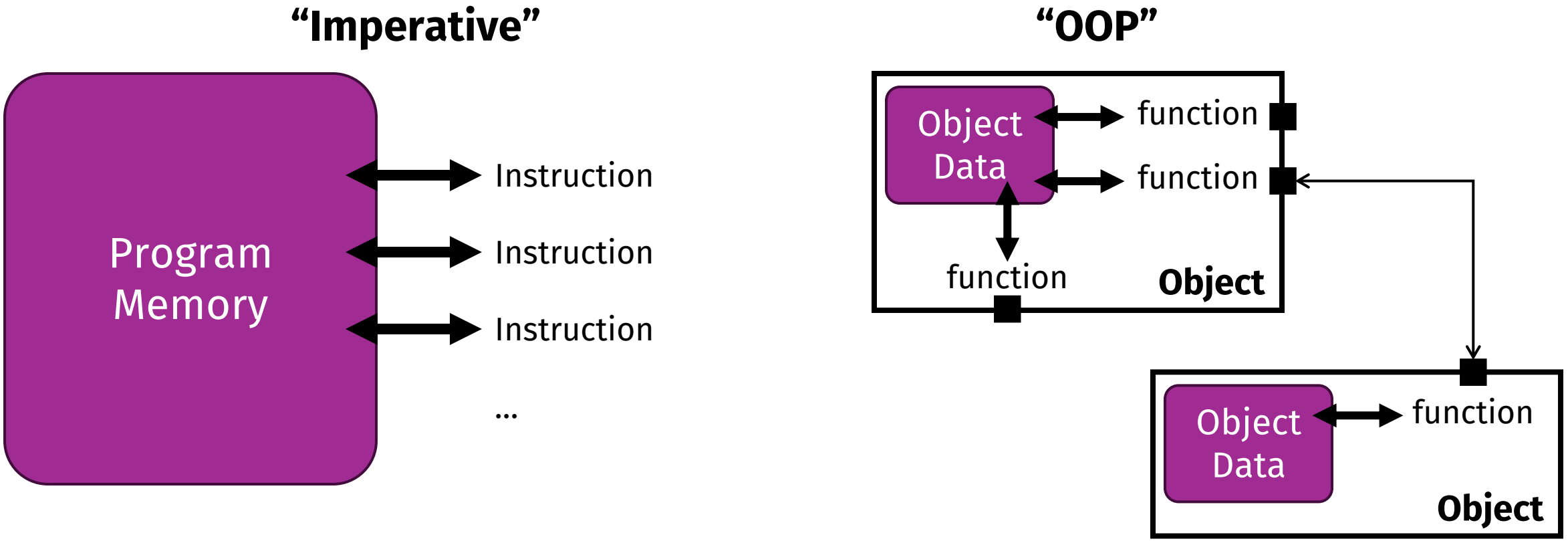
Example 2: User account





© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

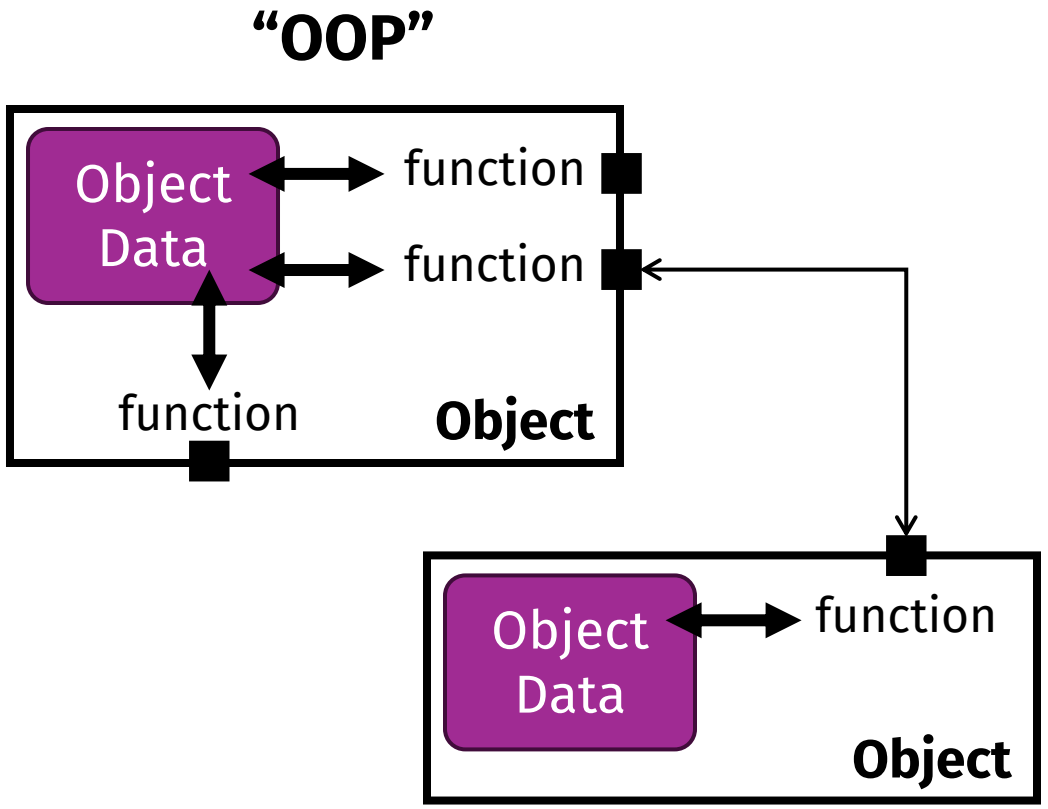
Object-Oriented Programming (OOP)



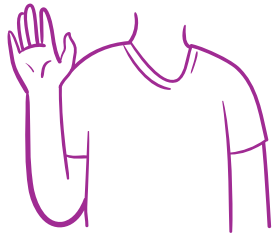
© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Object-Oriented Programming (OOP)

Decompose programs into a set of interacting **objects** that
(abstractions)
encapsulate data and functions
(information hiding)



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

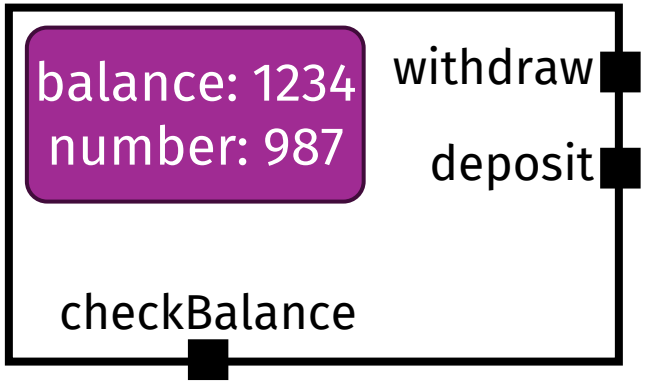


What's the difference between a class and an object?

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

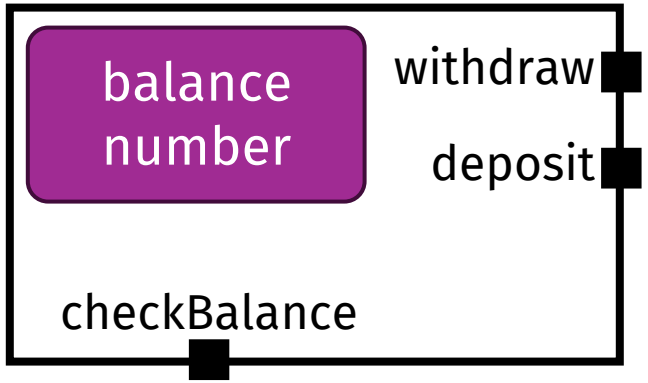
Classes and Objects

Object



Packaging of data together with functionality

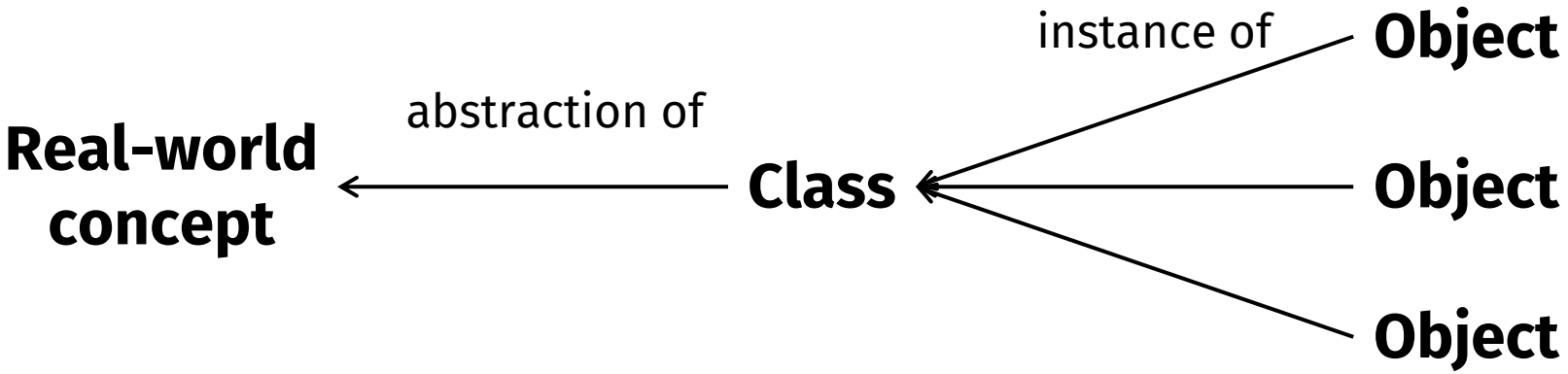
Class



Blueprint to define multiple objects with a similar form

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Classes and Objects



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Classes can represent...

Physical objects

- ✓ Client
- ✓ Bank

Functional abstractions

- ✓ Account
- ✓ Connection to a server

Cohesive sets of data

- ✓ Transaction information

Agents

- ✓ Foreign exchange calculator
- ✓ Transaction verification agent
- ✓ Authenticator

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Design Patterns



Reusable solution to common problems

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Example: Decorator

Problem: How to add behaviours to existing classes, without inheritance?

Solution: Use a Wrapper class that aggregates the base class, delegates methods to the base class, and adds the require behaviour. It’s possible to use multiple decorators to combine different behaviours.

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

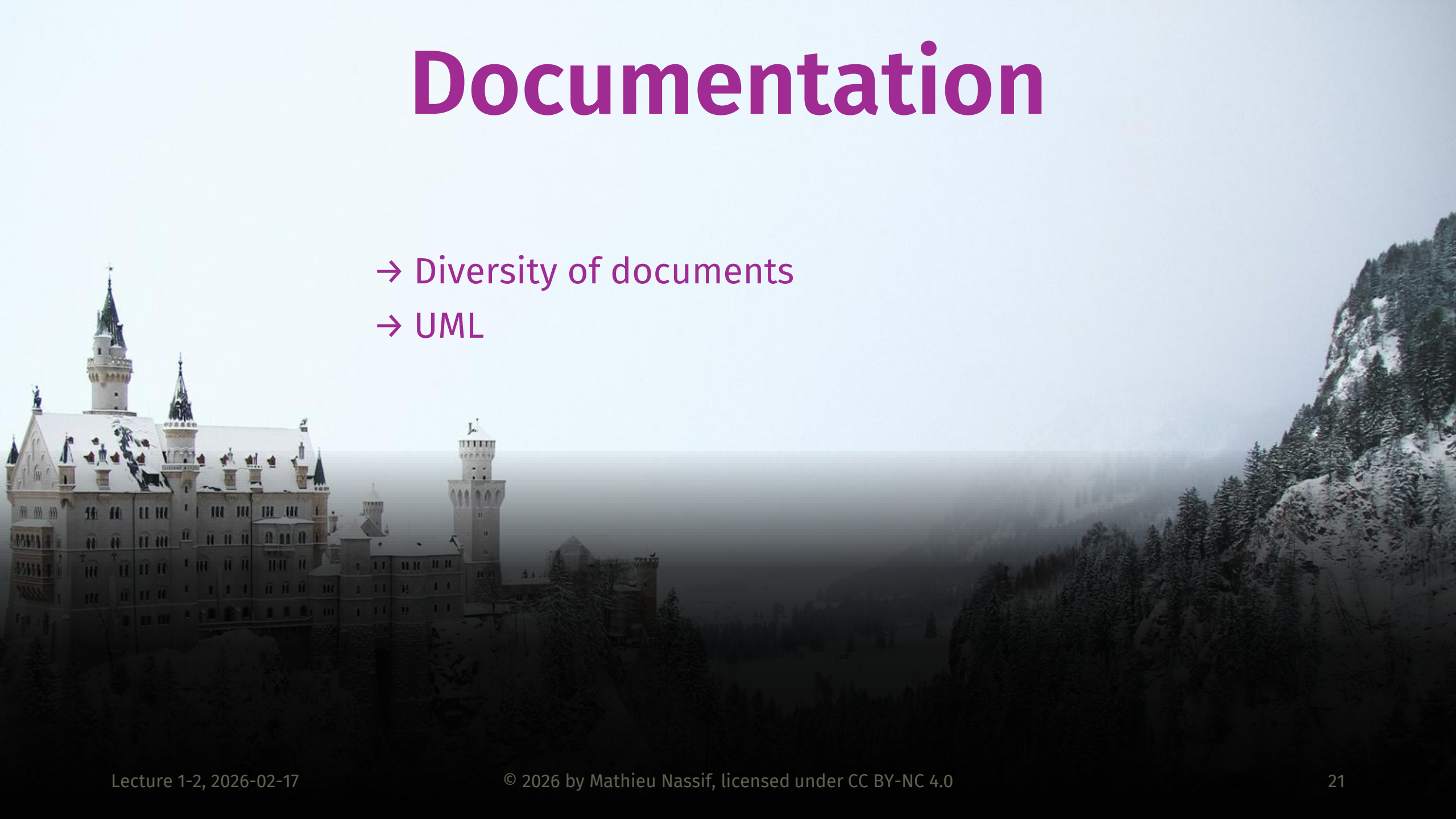
Designing Software



- Identified main components
- Designed structure of components
- Designed behaviour of components
- Identified critical parts and matched the level of details
- Verified and validated design

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Documentation



→ Diversity of documents

→ UML

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Have you written documentation?

Why?

What form?

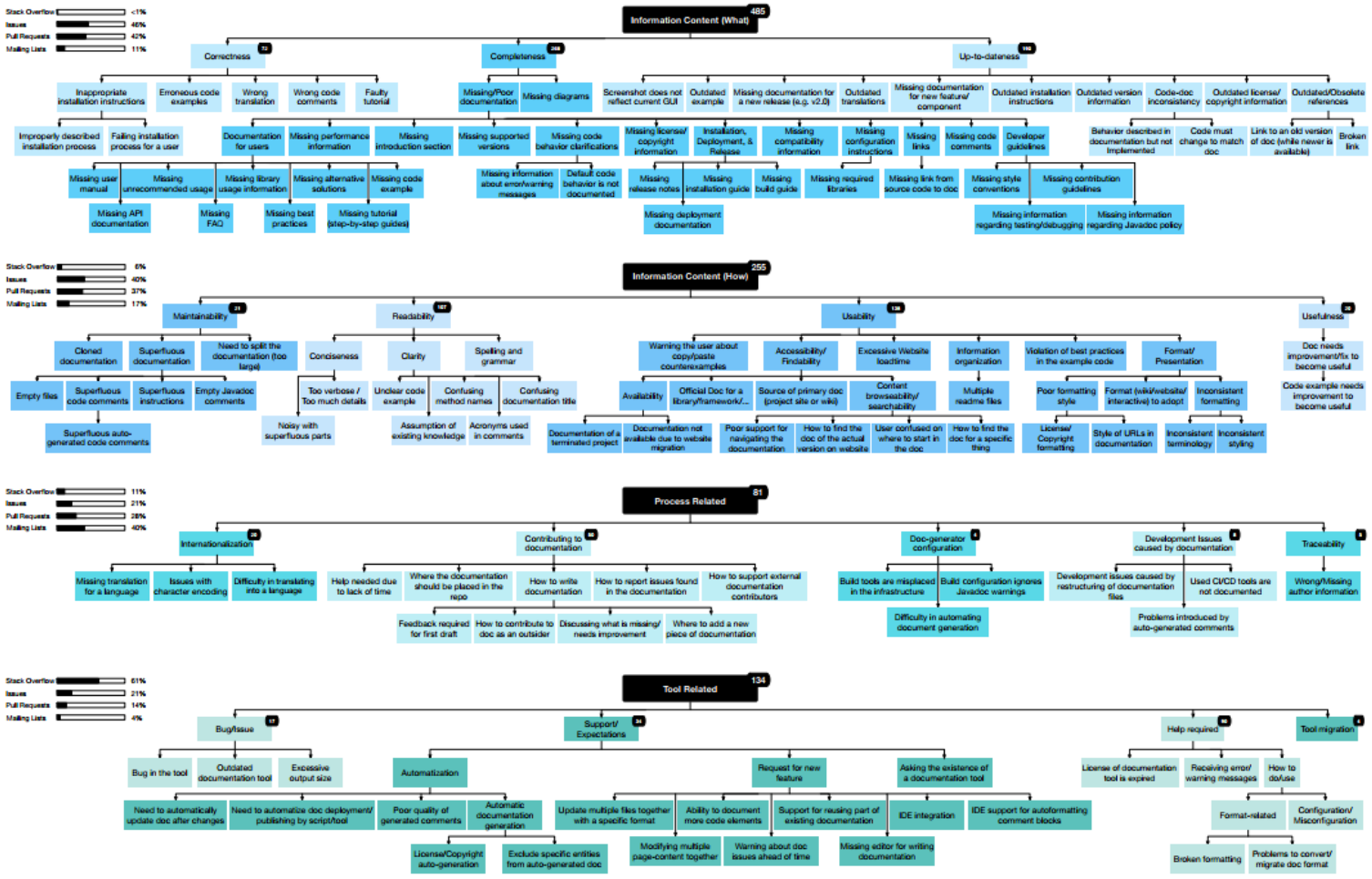
© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Types of documentation

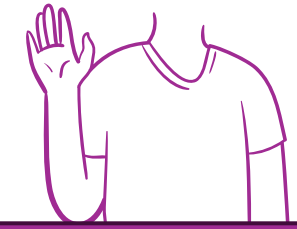
- Code comments
- API reference (docstrings, javadoc, …)
- Release notes
- User manual
- Roadmap
- Specification
- Architecture and design
- Contribution guidelines
- …

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Important, but overlooked



Incomplete
Inaccurate
Outdated
Too verbose
Too technical
Unclear
…



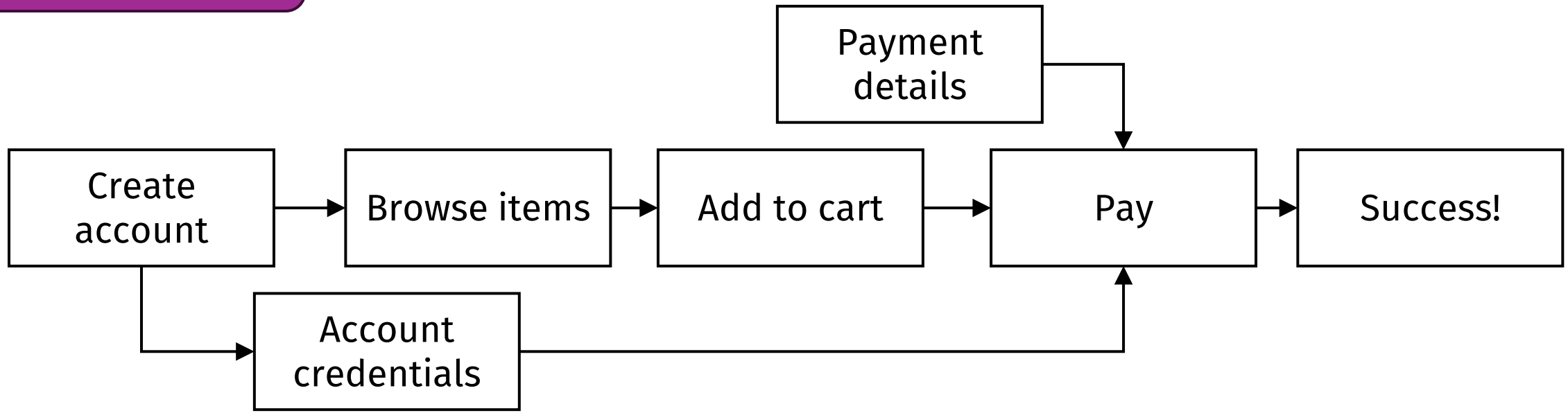
Why so many issues?

Aghajani et al., “Software Documentation Issues Unveiled.” ICSE, 2019

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

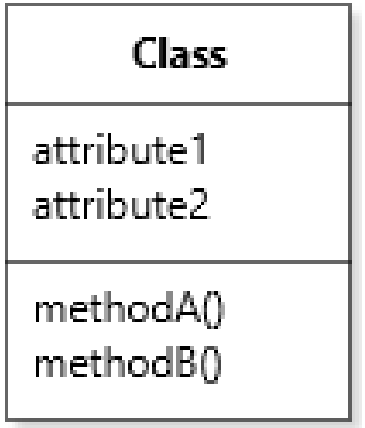
Designing with Diagrams

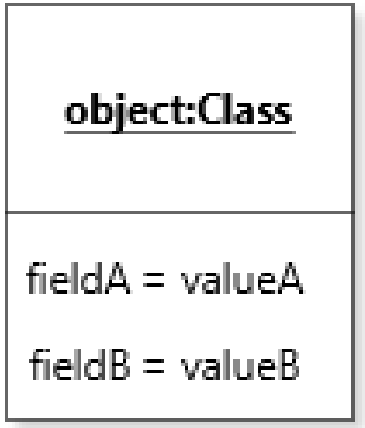




© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Unified Modeling Language

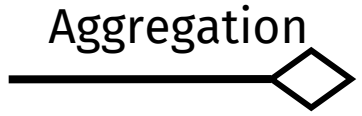








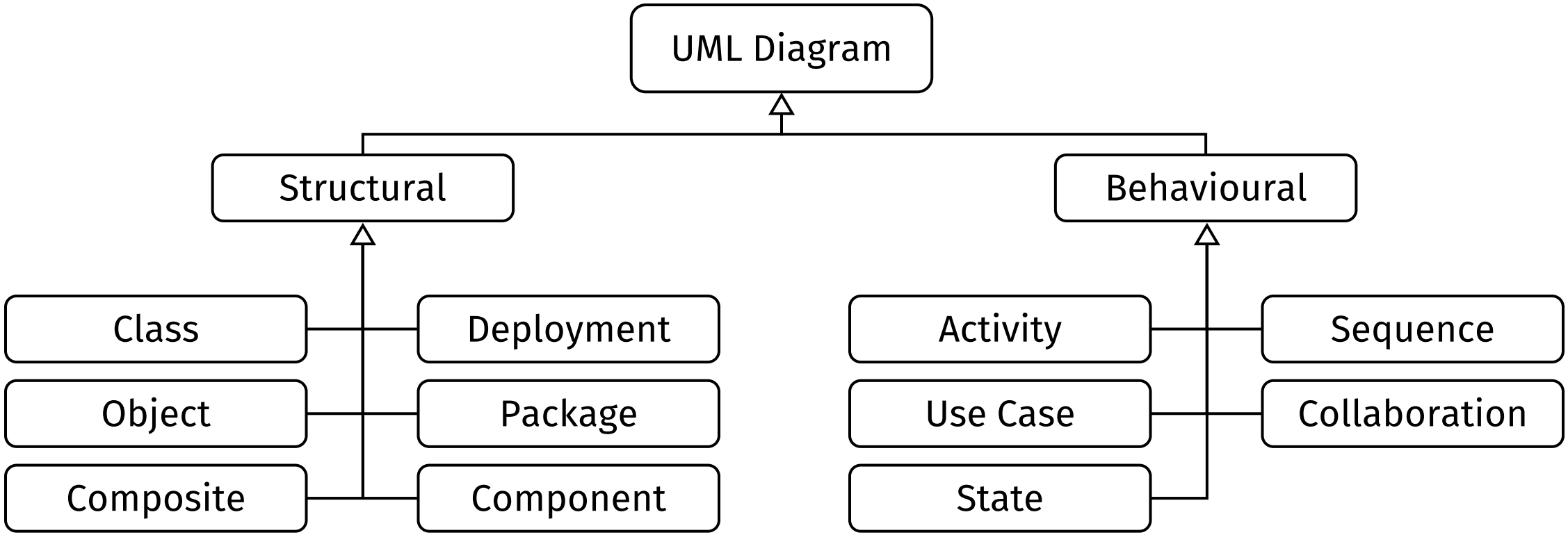




And more...

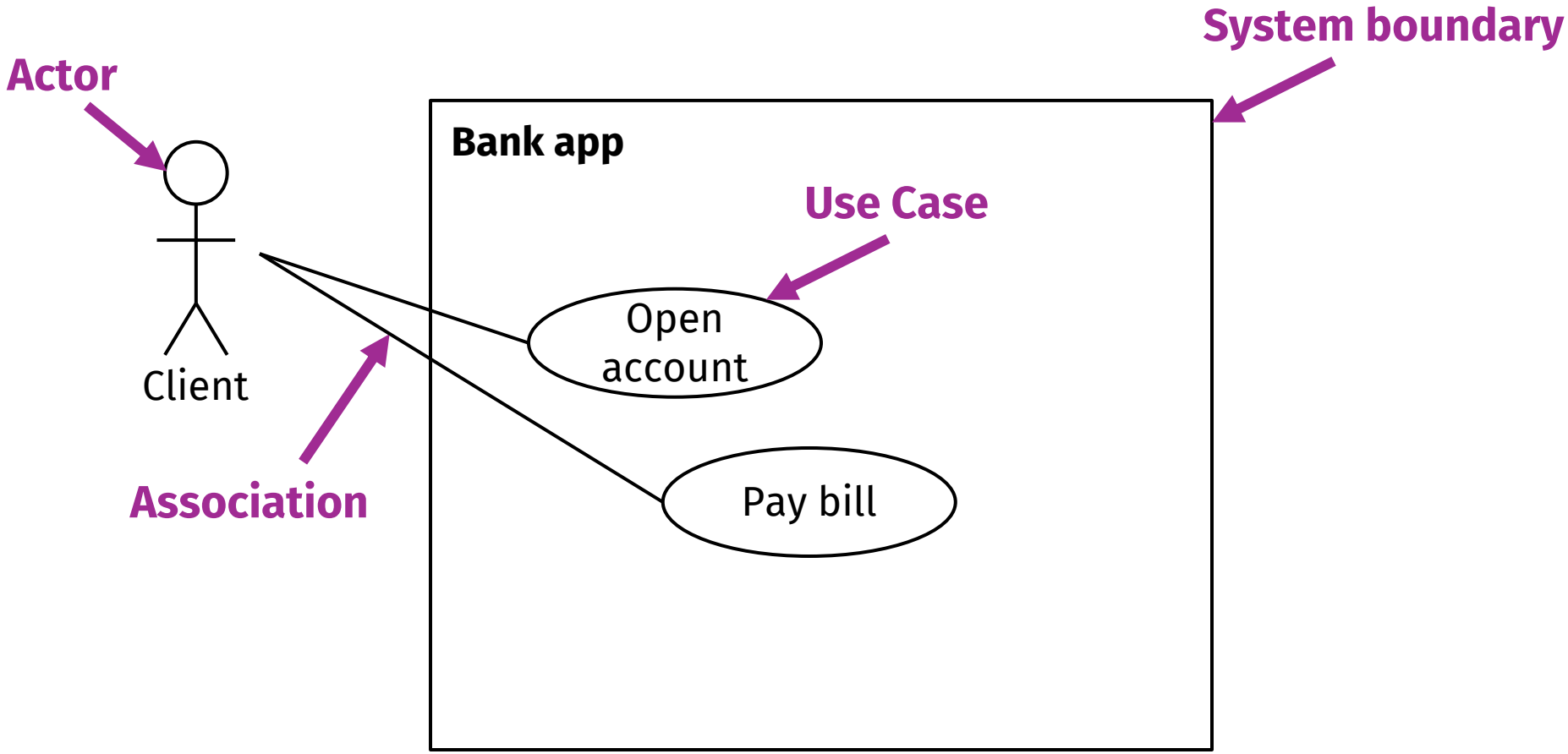
© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Types of UML diagrams



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Example: UML Use Case Diagram



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Example: UML Use Case Diagram

System boundary: what's supported by the system or not

Use Case: substantial actions that users can perform

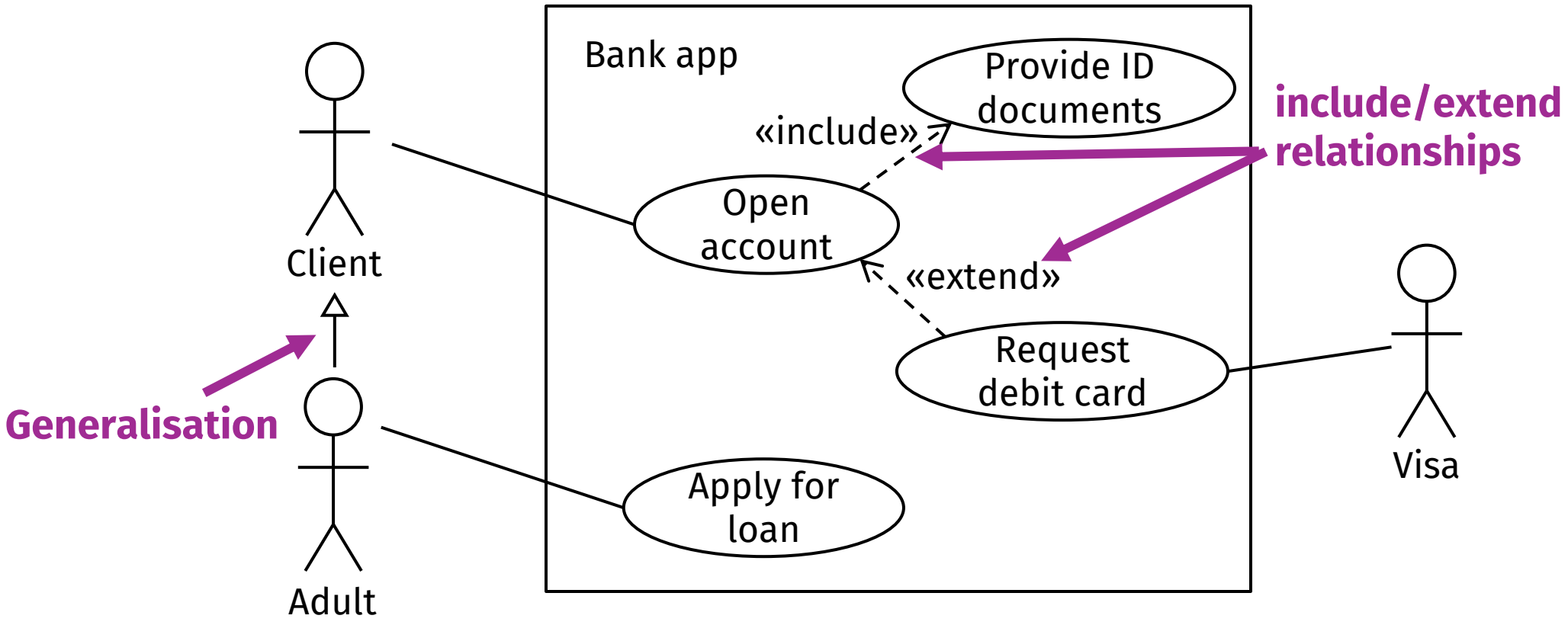
(functional requirements)

Actor: anyone interacting with the system

(not all stakeholders)

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Example: UML Use Case Diagram



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Example: UML Use Case Diagram

Actor generalization: can do what the parent can, and more

«inherit»: required use case to complete the base
(base use case incomplete without included use case)

«extend»: conditional or optional use case to add to the base
(base use case fully functional by itself)

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

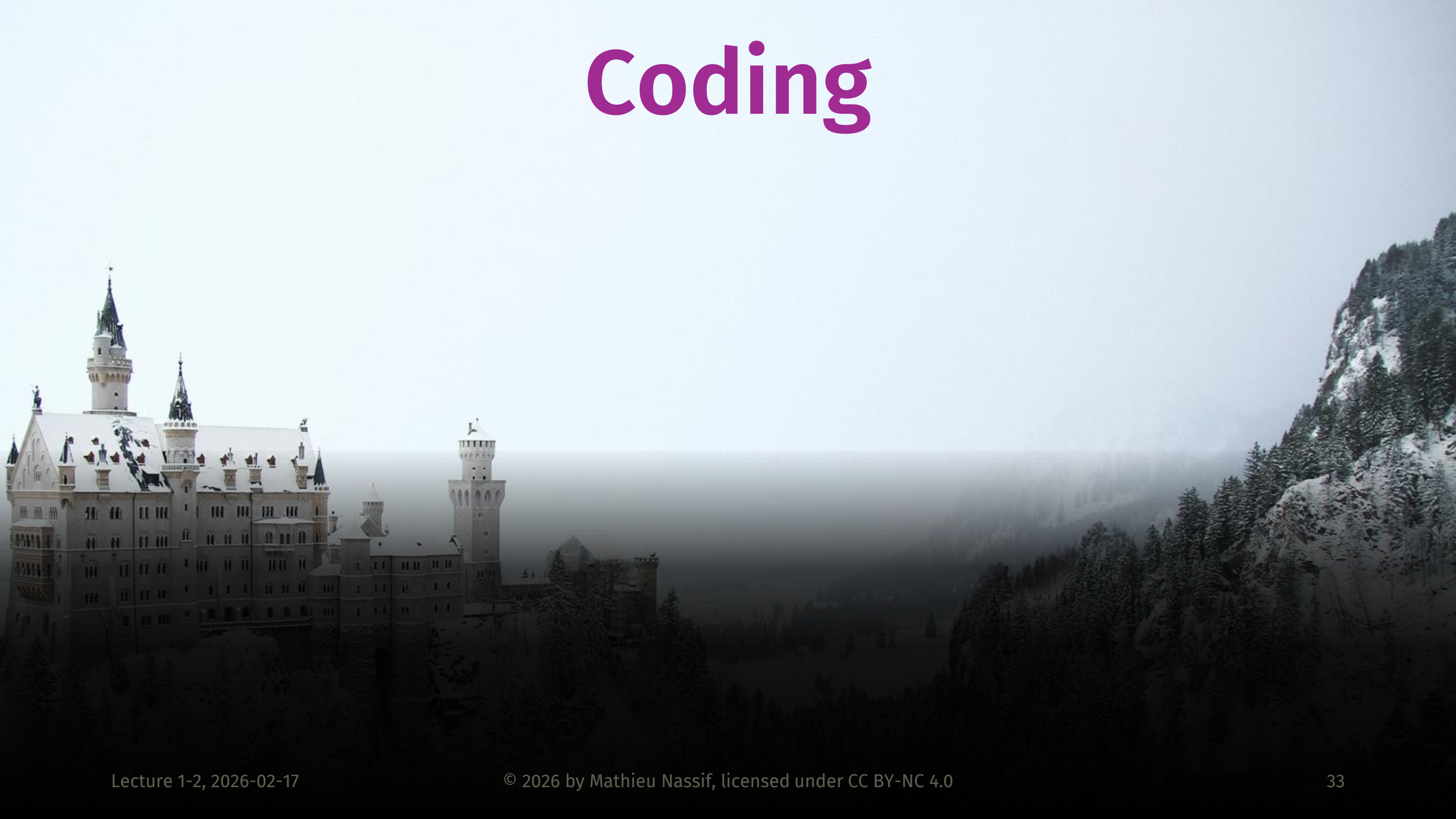
What’s not covered by Use Case diagrams?

- Sequence of actions, preconditions
- Success scenario vs error handling
- How to perform each action
- Alternative flows
- …

(must be documented elsewhere, possibly in text)

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Coding



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

(later)

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Testing



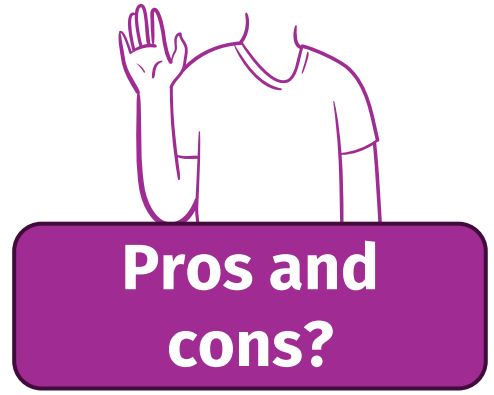
→ Automated testing

→ Scope of tests

→ Verification and Validation

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

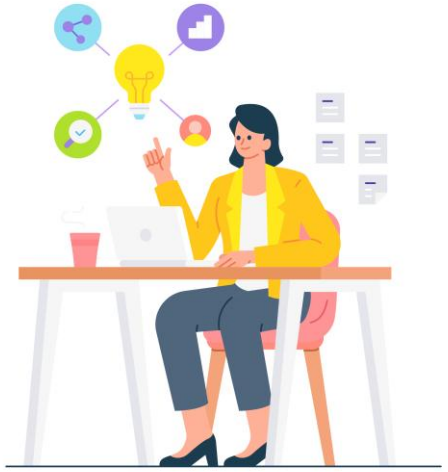
Different ways to test software



Meeting with client
Focus groups
User survey



Local run
Debugger
“Print” statements



Automated tests



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

What to test?

- Functional requirements
  - ✓ Individual functions (unit tests)
  - ✓ Interaction between components (integration tests)
  - ✓ Whole system (end-to-end tests, smoke tests)
  - ✓ Acceptance criteria (alpha/best tests)

- Quality requirements
  - ✓ Performance (load tests, stress tests)
  - ✓ Security (penetration tests)
  - ✓ Usability (accessibility tests, cross-platforms tests)

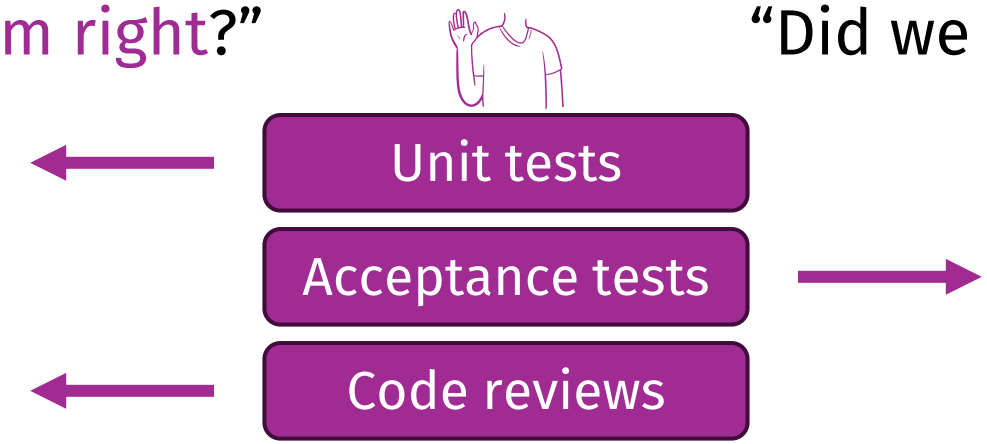
© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Ok, but *what* do we test?

Verification and **Validation**

Absence of bugs
Software and code quality
“Did we build the system right?”

Address client’s needs
Intended usage
“Did we build the right system?”



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

(later)

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Packaging and deploying software



→ Building software

→ CI/CD

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

You finished coding and testing. Now what?

How to quickly build a trimmed Linux kernel

This guide explains how to swiftly build Linux kernels that are ideal for testing purposes, but perfectly fine for day-to-day use, too.

The essence of the process (aka 'TL;DR')

[If you are new to compiling Linux, ignore this TLDR and head over to the next section below: it contains a step-by-step guide, which is more detailed, but still brief and easy to follow; that guide and its accompanying reference section also mention alternatives, pitfalls, and additional aspects, all of which might be relevant for you.]

If your system uses techniques like Secure Boot, prepare it to permit starting self-compiled Linux kernels; install compilers and everything else needed for building Linux; make sure to have 12 Gigabyte free space in your home directory. Now run the following commands to download fresh Linux mainline sources, which you then use to configure, build and install your own kernel:

```
git clone --depth 1 -b master \
  https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git ~/linux/
cd ~/linux/
# Hint: if you want to apply patches, do it at this point. See below for details.
# Hint: it's recommended to tag your build at this point. See below for details.
yes "" | make localmodconfig
# Hint: at this point you might want to adjust the build configuration; you'll
#   have to, if you are running Debian. See below for details
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Building software isn't always trivial

- Compiler options, optimizations, versions
- Cross-platform compilers, different targets
- Dependencies, and nested dependencies
- Auto-generated code
- Packaging considerations, what to include
- Build script configuration options
- High computational resource costs

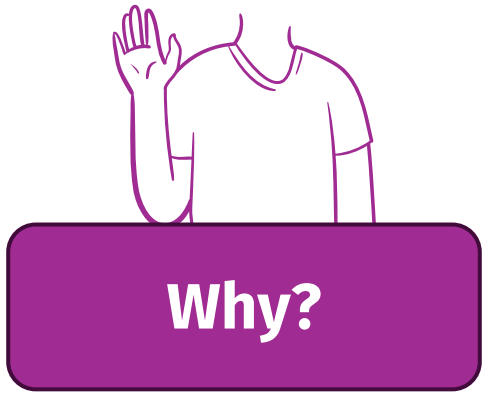
Goal
Reproduceable and reliable builds

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Continuous integration and deployment

Companies adopt CI/CD pipelines and
release tiny updates frequently,
often multiple times per day.

Lower risk



Quicker feedback

Easier to resolve problems

Avoid silos

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Maintaining and evolving software



→ Software decay

→ Types of maintenance

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

How does software go bad?



- Changing environment
- Changing needs
- OS compatibility
- Hardware evolution

- Complexity from new features
- Forgetting previous guidelines
- Evolving dependencies
- Deprecated dependencies
- New vulnerabilities found

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Types of maintenance

Bug report		Corrective maintenance
Feature request		Adaptive maintenance
Code improvement		Preventive maintenance

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

RECAP

→ Design, testing, deployment, maintenance

→ Documentation

→ UML



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0